CARNUS

Enseignement supérieur **RODEZ**

BTS CIEL — INFORMATIQUE ET RÉSEAUX

Contrôle d’accès, authentification et réservation

Octobre 2026

ÉTABLISSEMENT
CARNUS D’ENSEIGNEMENT SUPÉRIEUR
ENSEIGNEMENT SUPÉRIEUR

PROFESSEUR
K.B.
BTS CIEL — IR

Sommaire

Contrôle d’accès, authentification et réservation 1
Sommaire 2
1. Présentation de la tâche 3
1.1 Périmètre et objectifs fonctionnels 3
1.2 Interactions avec l'architecture globale 3
2. Analyse et Conception 4
2.1 Traitement des scénarios d'accès 4
2.2 Schéma de séquence de validation 5
3. Réalisation technique 6
3.1 Câblage et interfaçage matériel 6
4. Code source Firmware C++ 7
4.1 Inclusions des bibliothèques, configurations et variables globales 7
4.2 Fonction setup() — Initialisation du système 8
4.3 Fonction loop() — Détection et lecture de l'UID 9
4. Tests et Cahier de Recette 11
5. Cybersécurité et Bilans 12
5.1 Mesures de sécurité intégrées 12
5.2 Bilan technique et perspectives 12

1. Présentation de la tâche

Dans le cadre du Mini-Projet 2 (MP2), la Tâche 1 consiste à concevoir, câbler, programmer et intégrer un sous-système autonome de contrôle d'accès par badge RFID pour la salle 215. Ce système s'appuie sur un microcontrôleur ESP32 interconnecté à un lecteur RFID NXP MFRC522 et communique avec le serveur central Linux via une API REST JSON.

1.1 Périmètre et objectifs fonctionnels

- Interfaçage du lecteur MFRC522 avec l'ESP32 via le bus SPI (alimentation 3.3V).
- Lecture de l'identifiant unique (UID) des badges et cartes RFID MIFARE (13.56 MHz).
- Transmission réseau Wi-Fi sécurisée de l'UID à l'API du serveur central.
- Signalisation visuelle et sonore du statut de la demande d'accès (LED Verte, LED Rouge, Buzzer).
- Journalisation de toutes les tentatives d'accès dans la base de données centrale.

1.2 Interactions avec l'architecture globale

Le module RFID constitue le point d'entrée physique du système de contrôle d'accès. Il envoie ses requêtes d'autorisation à l'API centrale (Tâche 3), qui interroge la base de données SQL pour vérifier si l'utilisateur possède les droits d'accès sur le créneau horaire courant. La réponse détermine l'ouverture effective et met à jour l'état de la salle (occupée/libre).

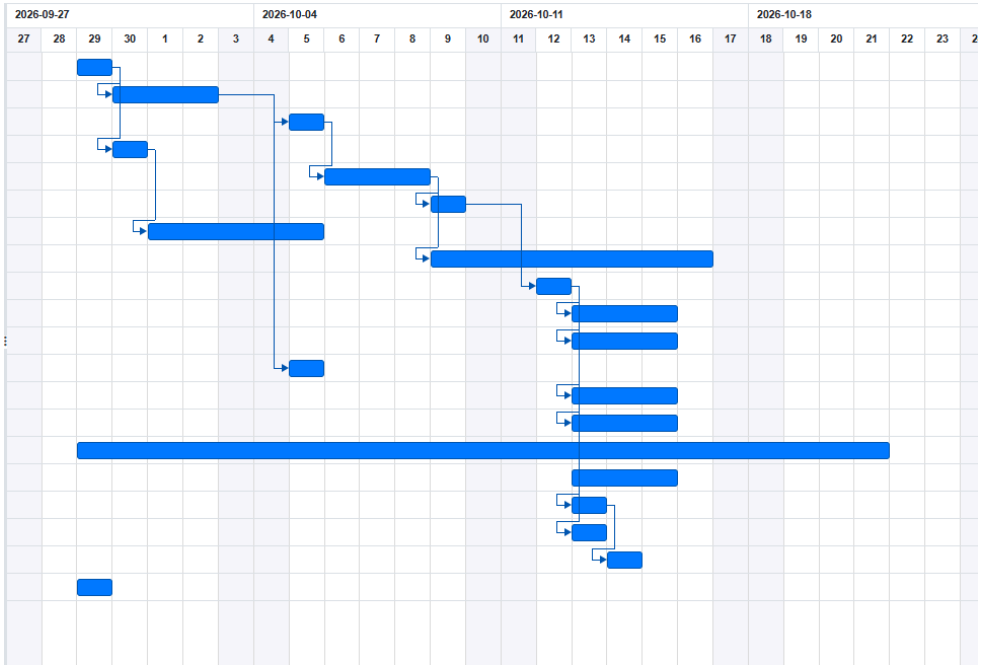
2. Analyse et Conception

2.1 Traitement des scénarios d'accès

Conformément au cahier des charges du MP2, le système gère quatre cas de figure distincts lors de la présentation d'un badge :

Scénario d'accès	Comportement du serveur / API	Signalisation locale (ESP32)
Accès Autorisé	Code HTTP 200 OK + enregistrement du log	LED Verte allumée (2s) + Bip court buzzer
Badge Inconnu	Code HTTP 403 Forbidden (Inconnu)	LED Rouge clignotante (2 bips longs)
Utilisateur non autorisé sur le créneau	Code HTTP 403 Forbidden (Non autorisé)	LED Rouge fixe (3s) + Bip long
Erreur réseau / Timeout API	Aucune réponse ou erreur HTTP 500	Clignotement alternatif LED Rouge/Verte

2.1 Gantt

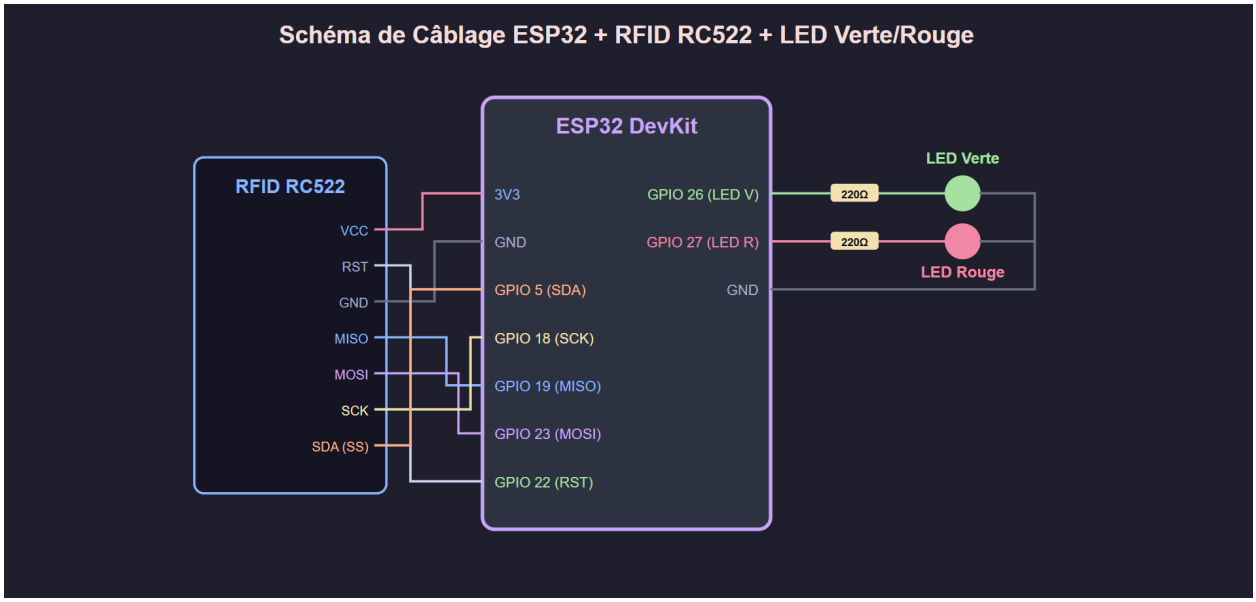


ID	Task Name	Start	End	Duration	Progress %	Dependency	Resources	Color	Notes
1	Analyse du besoin et choix du matériel (lecteur R…	2026-09-29	2026-09-29	1 day	0		Les deux		À confirmer
2	Câblage ESP32 + lecteur RFID	2026-09-30	2026-10-02	3 days	0	1FS	Embarqué		À confirmer
3	Firmware : lecture de l'UID	2026-10-05	2026-10-05	1 day	0	2FS	Embarqué		À confirmer
4	Contrat d'interface avec T3 (endpoint, JSON, cod…	2026-09-30	2026-09-30	1 day	0	1FS	Réseau		À confirmer
5	Communication ESP32 → API (Wi-Fi, requête H…	2026-10-06	2026-10-08	3 days	0	3FS	Embarqué		À confirmer
6	Gestion des 4 cas (autorisé, inconnu, refusé, err…	2026-10-09	2026-10-09	1 day	0	5FS	Embarqué		À confirmer
7	Journalisation des accès (logs)	2026-10-01	2026-10-05	3 days	0	4FS	Réseau		À confirmer
8	Sécurité : jeton, secrets hors du code, validation …	2026-10-09	2026-10-16	6 days	0	5FS	Réseau		À confirmer
9	Intégration avec T3 et test avec un vrai badge	2026-10-12	2026-10-12	1 day	0	6FS	Les deux		À faire
10	Phase de test : tableau de tests	2026-10-13	2026-10-15	3 days	0	9FS	Les deux		À faire
11	Fiches-recettes	2026-10-13	2026-10-15	3 days	0	9FS	Les deux		À faire
12	Schémas de câblage et schéma du flux RFID	2026-10-05	2026-10-05	1 day	0	2FS	Embarqué		À faire
13	Dépôt GitHub : code, configuration modèle, capt…	2026-10-13	2026-10-15	3 days	0	9FS	Les deux		À faire
14	Manuel technique	2026-10-13	2026-10-15	3 days	0	9FS	Réseau		À faire
15	Rapport technique T1 (10-12 pages)	2026-09-29	2026-10-21	17 days	0		Les deux		À faire
16	Contribution au rapport commun MP2 (5-6 pages…	2026-10-13	2026-10-15	3 days	0		Les deux		À faire
17	Conclusion et bilan du rapport	2026-10-13	2026-10-13	1 day	0	9FS	Les deux		À faire
18	Préparation de la présentation (5-6 min) et de la …	2026-10-13	2026-10-13	1 day	0	9FS	Les deux		À faire
19	FIN DU MP2 (jalon)	2026-10-14	2026-10-14	1 day	0	17FS	Les deux		À faire

3. Réalisation technique

3.1 Câblage et interfaçage matériel

Le module MFRC522 communique avec l'ESP32 via le bus bus synchrone SPI. L'alimentation du RC522 est impérativement délivrée sous 3.3V.



4. Code source Firmware C++

4.1 Inclusions des bibliothèques, configurations et variables globales

```
// Inclusion des bibliothèques nécessaires
#include <SPI.h>          // Gestion de la communication bus SPI avec le module RFID
#include <MFRC522.h>      // Gestion du lecteur de carte RFID RC522
#include <WiFi.h>         // Gestion de la connexion Wi-Fi de l'ESP32
#include <HTTPClient.h>   // Permet d'envoyer des requêtes HTTP (POST) au serveur
#include <ArduinoJson.h>  // Permet de créer et manipuler du format JSON

// Définition des broches de connexion matérielle (GPIO)
#define SS_PIN    21      // Broche SDA / Chip Select du lecteur RFID
#define RST_PIN   22      // Broche Reset du lecteur RFID
#define LED_GREEN 2       // Broche de la LED verte (Accès autorisé)
#define LED_RED   4       // Broche de la LED rouge (Accès refusé)
#define BUZZER    15      // Broche du buzzer (Signal sonore)

// Initialisation de l'objet lecteur RFID avec ses broches
MFRC522 rfid(SS_PIN, RST_PIN);

// Identifiants du réseau Wi-Fi
const char* ssid = "Wifi_Salle_215";       // Nom du réseau Wi-Fi
const char* password = "PasswordSecurise";  // Mot de passe du réseau Wi-Fi

// Adresse IP du serveur Linux et endpoint de l'API REST
const char* apiEndpoint = "http://172.16.10.245/api/v1/access";
```

4.2 Fonction `setup()` — Initialisation du système

```
void setup() {
  // Initialisation de la liaison série pour afficher les logs sur le PC (115200 bauds)
  Serial.begin(115200);
  // Initialisation du bus SPI et du lecteur RFID MFRC522
  SPI.begin();
  rfid.PCD_Init();
  // Configuration des broches des LED et du Buzzer en mode sortie
  pinMode(LED_GREEN, OUTPUT);
  pinMode(LED_RED, OUTPUT);
  pinMode(BUZZER, OUTPUT);
  // Démarrage de la connexion Wi-Fi
  Serial.print("Connexion au Wi-Fi");
  WiFi.begin(ssid, password);
  // Boucle d'attente jusqu'à ce que la connexion Wi-Fi soit établie
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);         // Pause de 0,5 seconde entre chaque tentative
    Serial.print(".");
  }
  // Confirmation de connexion et affichage de l'IP attribuée à l'ESP32
  Serial.println("\n[RESEAU] Wi-Fi connecte !");
  Serial.print("[RESEAU] Adresse IP de l'ESP32 : ");
  Serial.println(WiFi.localIP());
}
```

4.3 Fonction `loop()` — Détection et lecture de l'UID

```
void loop() {
  // Vérifie si un badge RFID est présent devant le lecteur
  if (!rfid.PICC_IsNewCardPresent() || !rfid.PICC_ReadCardSerial()) {
    return; // Si aucun badge n'est détecté, on sort et on réessaie à la boucle suivante }
  // Variable texte pour stocker l'UID sous forme d'une chaîne
  String uidStr = "";
  // Parcours des octets de l'UID du badge lu pour les convertir en Hexadécimal
  for (byte i = 0; i < rfid.uid.size; i++) {
    // Ajoute un '0' devant si la valeur est inférieure à 16 (0x10) pour garder 2 caractères
    uidStr += String(rfid.uid.uidByte[i] < 0x10 ? "0" : "");
    uidStr += String(rfid.uid.uidByte[i], HEX); }
  // Met toutes les lettres de l'UID en majuscules (ex: "A1B2C3D4")
  uidStr.toUpperCase();
  // Affichage de l'UID lu sur le moniteur série
  Serial.print("[RFID] Badge detecte - UID : ");
  Serial.println(uidStr);
  // Appel de la fonction pour envoyer l'UID au serveur Linux
  envoyerRequeteAccess(uidStr);
  // Halte de la carte RFID pour éviter de la relire en boucle
  rfid.PICC_HaltA();
  rfid.PCD_StopCrypto1();
  // Pause de 1 seconde avant la prochaine lecture
  delay(1000); }
```

```
void envoyerRequeteAccess(String uid) {
  // Vérifie que le Wi-Fi est toujours bien connecté
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;

    // Prépare la requête HTTP vers l'URL du serveur (172.16.10.245)
    http.begin(apiEndpoint);

    // Spécifie au serveur que l'on envoie des données au format JSON
    http.addHeader("Content-Type", "application/json");

    // Création du document JSON contenant l'UID
    StaticJsonDocument<128> doc;
    doc["uid"] = uid;

    // Conversion du JSON en chaîne de texte
    String requestBody;
    serializeJson(doc, requestBody);

    // Envoi de la requête HTTP POST contenant le JSON
    int httpCode = http.POST(requestBody);

    // Traitement de la réponse du serveur
    if (httpCode == 200) {
      // Cas 1 : Accès autorisé par le serveur
      Serial.println("[ACCES] Autorise (HTTP 200)");
      digitalWrite(LED_GREEN, HIGH);  // Allume la LED verte
      digitalWrite(BUZZER, HIGH);     // Déclenche le signal sonore
      delay(2000);                    // Maintient le signal pendant 2 secondes
      digitalWrite(LED_GREEN, LOW);   // Éteint la LED verte
      digitalWrite(BUZZER, LOW);      // Arrête le buzzer
    }
    else {
      // Cas 2 : Accès refusé (Badge inconnu ou non autorisé)
      Serial.print("[ACCES] Refuse - Code HTTP : ");
      Serial.println(httpCode);
      digitalWrite(LED_RED, HIGH);    // Allume la LED rouge
      delay(2000);                    // Maintient la LED rouge 2 secondes
      digitalWrite(LED_RED, LOW);     // Éteint la LED rouge
    }

    // Fermeture propre de la connexion HTTP
    http.end();
  }
  else {
    // Si la connexion Wi-Fi est perdue
    Serial.println("[ERREUR] Wi-Fi deconnecte ! impossible d'envoyer la requete.");
  }}
```

4. Tests et Cahier de Recette

Afin de valider la conformité du système par rapport aux exigences du sujet, les fiches-recettes suivantes ont été exécutées :

ID Test	Fonctionnalité testée	Conditions / Actions	Résultat attendu	Statut
T01	Lecture de l'UID RFID	Passage d'un badge MIFARE 1K devant le RC522	UID correctement extrait et affiché sur la console série	**OK**
T02	Accès Autorisé	Badge enregistré avec réservation active	HTTP 200 reçu, LED Verte allumée pendant 2 sec	**OK**
T03	Accès Refusé (Inconnu)	Badge non répertorié dans la BDD	HTTP 403 reçu, LED Rouge allumée	**OK**
T04	Gestion déconnexion Wi-Fi	Coupure du point d'accès Wi-Fi	Avertissement visuel local et tentative de re-connexion	**OK**

5. Cybersécurité et Bilans

5.1 Mesures de sécurité intégrées

- Validation et assainissement des entrées JSON côté serveur pour contrer les injections SQL.
- Exclusion des mots de passe Wi-Fi et identifiants du dépôt Git (utilisation de fichiers d'environnement `.env`).
- Préparation de l'intégration du protocole HTTPS (TLS) pour sécuriser les requêtes POST.

5.2 Bilan technique et perspectives

La Tâche 1 du Mini-Projet 2 a permis de valider l'ensemble des objectifs fixés. La chaîne complète (lecture matérielle SPI, communication REST et traitement serveur) est opérationnelle. La solution est facilement évolutive pour intégrer des fonctionnalités futures comme un second lecteur pour la sortie ou une commande de gâche électrique.